CODERCO · AI / MLOPS SERIES

Evaluation & Observability

05

how you know it works, and what it costs you

Mostly live. 16 slides, 7 demos, one running system.

Craig Li · ~50 minutes · Session 5 of 9

Why your monitoring stays green

four assumptions your existing observability rests on, and why none of them hold here

A normal service fails loudly

Returns HTTP 200 with a fluent, confident, wrong answer. No exception, no stack trace, nothing to alert on.

A normal service has a deterministic contract

Same input, different output. You cannot assert the result mechanically, so “it worked when I tried it” is not evidence and one bad example is not a bug report.

A request costs what the last one cost

Cost varies per request with tokens, and cost and latency are not reliably correlated — so a latency dashboard alone will not surface your expensive requests.

Quality is fixed at deploy

It drifts while the code stands still: the corpus moves, the questions move, the provider updates the model.

Uptime, latency and error rate stay green through every one of these.

Four questions. Most teams instrument one.

Is it up? latency, errors, throughput	you already have this	solved
Is it any good? groundedness, citations, refusals	most teams guess	today
What does it cost? per request, per conversation, per intent	nobody attributes it	today
Can you prove it? who asked, what it read, how the answer was produced	and a deadline is coming	today

Production AI needs an auditable decision trail before regulation forces it — and for Annex III high-risk employment AI, that deadline is 2 December 2027.

DEMO 1/7

One question, fully instrumented

▷ **make present → type a question, press Ask**

TTFT	time until the user sees anything — the “is it broken?” number
ITL, mean and p95	the gap between tokens — the “why is it so slow?” number
the stage bar	classify · retrieve · generate, side by side
cost, tokens, citations	and citations that resolve to provisions, not page numbers

In this workload retrieval is a rounding error on latency — and it still drives most of the quality decisions.

The two numbers a “latency” chart hides

TTFT grows with input + upstream work

Everything before the first token: your retrieval and assembly, queueing, and the model reading the whole prompt. Every extra chunk makes the start slower.

the direct cost of “retrieve more to be safe”

ITL degrades under saturation

Often steady in isolation, and frequently worsens as the serving fleet saturates — a useful model-serving capacity signal alongside CPU/GPU utilisation.

the stutter users complain about

```
e2e  ≈  TTFT  +  ITL × (output_tokens − 1)
```

the gap to measured e2e is non-generation overhead: orchestration, network, post-processing, client

Different causes, different owners, different fixes. One p95 number cannot tell you which moved.

DEMO 2/7

The incident you cannot see from one graph

Baseline · 24 then Incident · 18

TTFT p50	jumps — read the number off the panel, not off this slide
citation coverage	collapses toward zero, in the same window
separately	each looks like noise a busy team closes as “no repro”
together	they point at one story — and the TRACE is what confirms which model answered

Quality and latency on ONE time axis. On two dashboards owned by two teams, this is two closed tickets.

Two habits that make the panels honest

1 · never ONLY a mean (illustrative)

`groundedness  mean = 0.92`

Green. Above threshold. Nobody paged.

`groundedness  p10  = 0.45`

The worst tenth are barely supported — and answered with identical confidence.

2 · a trace, not just a log

`agent.request`	CHAIN
`retrieve`	RETRIEVER
`assess`	EVALUATOR
`generate`	LLM

OpenInference span kinds are what make a trace queryable rather than a pile of strings — distinct from OTel's SpanKind

A steady 0.85 with no catastrophes beats a 0.92 average hiding a disaster tail.

Same argument as p95 latency, which you already accept. Metrics say it is getting worse; the trace says why THIS one was wrong.

Watch the trend too: a sustained slide should trigger investigation even while the score is still above threshold.

DEMO 3/7

Reading one request in Phoenix

make phoenix → open the trace for the question you just asked

the span tree	classify · retrieve · assess · generate — no per-node tracing code, once instrumentation is configured
retriever span	the documents it returned, their scores, the index version
assess span	the sufficiency score against its threshold — a decision, recorded
then kill Phoenix	re-run: the agent still answers. Spans become no-ops.

Observability must never be able to take down the thing it observes.

The ruler, and the bug it found

one row of the golden set

```
{"id":"g13",
 "question":"What does section 18
             say about
             bereavement leave?",
 "must_cite":["s.18"]}
```

why it survives a model upgrade

- **asserts CITATIONS, not prose**
  and asks what a provision SAYS, not what you are entitled to
- **stratified by intent**
  a fix to one topic cannot mask a break in another
- **includes out-of-scope cases**
  refusing correctly is a behaviour worth testing

g17 — a valid question, refused

“Can a confidentiality clause cover harassment complaints?”

The Act says “contractual duties of confidentiality”. The user says “clause”. Sufficiency lands just under the gate — so a valid question gets declined.

Delete the case and the bug ships. Tune until it passes and you have fitted the gate to one example. Mark it KNOWN and assert the list does not GROW.

A suite that has to be perfect gets its failing cases quietly deleted.

DEMO 4/7

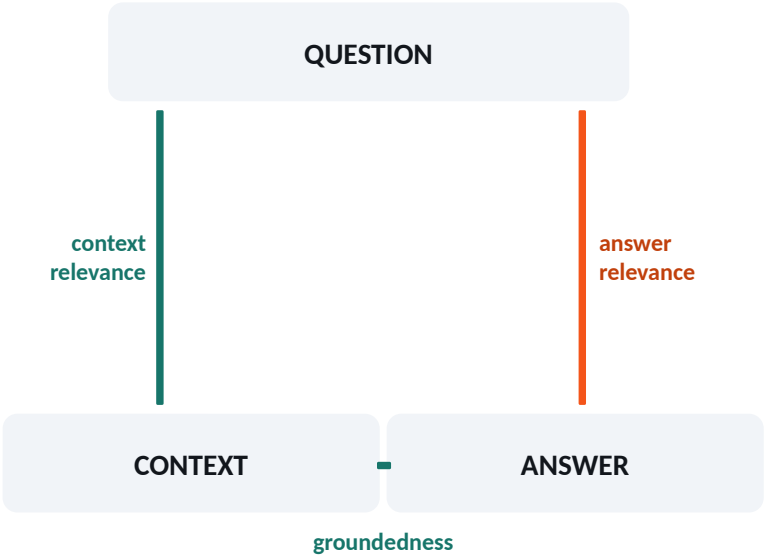
The gate

Run the CI gate

the whole suite	offline, under a minute, no API key, no flake
structural only	citations, refusals, routing, telemetry fields, the audit chain
the index is rebuilt	so a change to the chunker is actually exercised
break it live	delete the citation instruction from the prompt and re-run

Nothing here asks a model for an opinion — which is exactly why it may block a merge.

Three questions about one answer



low ctx-rel → stop editing the prompt

Context relevance

did we retrieve material capable of answering this?

RETRIEVAL is the suspect

Groundedness

is every claim supported by that material?

GENERATION is the suspect

Answer relevance

does it address what was actually asked?

question ↔ answer alignment

\+ citation coverage — is each claim attributable?

They isolate stages. High context relevance with low groundedness is a generation bug.

DEMO 5/7

Evaluate the evaluator

Calibrate the judge

12 clean examples	a perfect Cohen's kappa. Ask the room: would you trust it now?
+ 4 realistic ones	kappa falls. Read both numbers off the screen.
ground AND citations	gating on both recovers most of it — two complementary signals can beat either alone
case c13	a correct paraphrase scores zero. A lexical judge cannot see paraphrase.

The judge did not get worse when kappa fell. The test got honest.

What changes while your code stands still

THE QUESTIONS MOVE

Corpus drift
the source documents changed
→ re-index; date filters

Intent drift
the QUESTIONS moved
→ check coverage first

Performance drift
the scores fell
→ could be either, or the model

Intent mix moved AND quality fell together? Suspect a coverage gap, not a regression. Fix content, not temperature.

THE BILL MOVES

generation · input
the chunks you chose to send — every call, forever

generation · output
materially dearer per token than input on many models — brevity is a cost control

trace storage
cheap each, enormous in aggregate

vector DB + embedding
amortised per request

ranked by weight on a typical request — the live split is on the dashboard

The judge is on this list too — it is an LLM call. Scoring every request can materially increase the bill, which is why production evaluation is usually sampled.

prices live in one dated table in config.py — change it and every cost figure downstream moves

Neither of these is visible in code review. Both are visible in the metrics you already log.

DEMO 6/7

Drift, then the same traffic at two prices

Shift intents · 20 → Drift report then Reprice · Haiku / Sonnet

PSI + new-intent check	the known mix shifts, and unseen intents are flagged separately
not a bug report	it says the QUESTIONS changed — a content investigation
Haiku vs Sonnet	identical recorded tokens, a materially bigger bill
in THIS workload	generation INPUT dominates — the retrieved context you send, not the answer

Recorded tokens plus a price table is a cost model. No new instrumentation.

Explainability — and making edits detectable

EU AI Act, Article 12

Requires high-risk systems to technically allow automatic recording of events over the system's lifetime. It is a LOGGING CAPABILITY duty.

Certain employment uses are listed in Annex III; where classified high-risk, these requirements apply from 2 Dec 2027.

a practical audit record

engineering design — not an Article 12 field list

who	actor, tenant, role, lawful basis
what	the question; answered or refused
how	index + document VERSIONS, model, prompt
integrity	answer hash + hash of the record before

Reference versioned documents

Store ids, VERSION and hash — not a copy of the text. An id plus a hash tells you the source changed; the version is what lets you reconstruct what it said.

Define BOTH retention boundaries

AI ACT — logs you control from a high-risk system: appropriate to purpose, generally at least 6 months (Arts. 19 / 26).
GDPR — no longer than justified for that purpose. A principle, not one universal maximum.

Tamper-EVIDENT, not tamper-proof — and the log is now sensitive.

A hash chain catches a local edit; someone who can rewrite the whole store can recompute every hash, so sign or anchor it elsewhere. And redact at capture, not in the UI — it holds user questions and retrieved text.

RAG expands the model's knowledge boundary. Your trace store expands your data-protection boundary.

DEMO 7/7

Tamper with the audit log — then where to start

Tamper with the audit log

before	chain intact — every record verified from genesis
edit one field	in the FIRST record — a decision from an hour ago
after	CHAIN BROKEN at record 0 — a local edit is detected. Anchor the chain elsewhere to survive a full-store rewrite.

Monday:

0 one structured log line per request
1 TTFT + ITL as percentiles
2 30 golden questions from real traffic
3 deterministic checks in CI
4 a judge — then calibrate it
5 cost per request, attributed

Step 0 takes an afternoon and everything else needs it. · Session 6: securing an agent — and the trail you just built is how you catch it.